

1 Abstract

This report details the reconstruction and quantitative analysis of chromatographic data derived from a large-scale dataset containing 1.08 million records. The primary objective was to isolate 11 distinct logical experimental batches, defined by the ‘run’ identifier, to ensure valid inter-batch comparisons. Raw data underwent rigorous preprocessing, including a volume offset correction (+33.8) to eliminate negative cumulative volume artifacts and Asymmetric Least Squares (AsLS) baseline correction to mitigate high-frequency noise. Quantification was performed using weighted peak area calculations, where signal intensity was multiplied by fraction volume to account for varying elution volumes. Due to severe data missingness (>60%) for process parameters such as ‘chromatography_stage’ and ‘Conc_B_ml’, correlations with these variables were excluded in favor of ranking compounds based on total weighted peak area. The analysis successfully reconstructed individual chromatograms and established a robust ranking of the 11 runs based on compound abundance.

2 Introduction

Chromatographic analysis of complex mixtures often involves the collection of data in discrete fractions, which must be aggregated to reconstruct the continuous elution profile. In this study, a dataset comprising 1.08 million records was analyzed. A critical challenge identified was the discrepancy between sequential identifiers (‘run_no’) and logical batch identifiers (‘run’), where 11 distinct experimental groups were embedded within a larger sequence. Furthermore, raw cumulative volume data exhibited a systematic negative offset of -33.8 ml, likely due to instrument initialization delays or calibration drift. Process variables critical for understanding chromatographic behavior, such as the chromatography stage and concentration of buffer B (‘Conc_B_ml’), were largely missing (60-75% missingness), rendering standard correlation analyses infeasible without imputation. This report outlines the methodology employed to correct data integrity issues, reconstruct chromatograms via weighted aggregation, and rank the experimental runs based on the total abundance of detected compounds using the UV_260 nm signal.

3 Methodology

The data analysis workflow consisted of three primary stages. First, data integrity and baseline correction were applied to the raw ‘chromatography_combined.csv’ file. Cumulative volumes were corrected by adding a constant offset of 33.8 ml to eliminate negative values. The UV_260 nm signal underwent Asymmetric Least Squares (AsLS) baseline correction using a lambda parameter of 1,000,000 and an asymmetry parameter of 0.00001 to smooth the signal while preserving peak shapes. Second, weighted aggregation and peak area calculation were performed. Data was aggregated by the logical ‘run’ identifier and ‘Fraction_number’. The weighted peak area for each fraction was calculated as the product of the mean corrected UV signal and the corresponding fraction volume. A fallback procedure was implemented for records with missing precise volume bounds, utilizing the standard ‘fraction_volume’ variable. Third, compound ranking and correlation assessment were conducted. Runs were ranked by their total weighted peak area. Correlation assessments between the signal and process variables (‘chromatography_stage’, ‘Conc_B_ml’) were attempted; however, due to missingness exceeding 60%, these calculations were explicitly skipped to avoid statistical bias.

4 Results and Discussion

The reconstruction of individual chromatograms from the aggregated fraction data successfully visualized the elution profiles for all 11 logical runs. As illustrated in Figure 1, the weighted peak area plots display the signal intensity normalized by fraction volume across the elution range (0-490 fractions). The subplots reveal distinct elution profiles for each run, demonstrating the effectiveness of the aggregation and weighting methodology in resolving the chromatographic behavior despite the high noise levels present in the raw data. The AsLS baseline correction effectively removed the baseline drift, allowing for clear identification of peak maxima. The variation in peak shapes and areas across the 11 runs suggests differences in the experimental

conditions or sample composition within these batches. The exclusion of correlation analysis with process parameters, due to the high missingness of ‘chromatography_stage’ and ‘Conc.B_ml’, limits the ability to directly link specific process variables to the observed chromatographic variations in this specific dataset. However, the ranking of runs by total peak area provides a robust metric for comparing the overall yield or concentration of the dominant compounds across the different experimental batches.

5 Conclusion

This study successfully reconstructed 11 distinct chromatographic profiles from a massive, noisy dataset by correcting volume offsets and applying robust baseline correction techniques. The use of weighted peak area aggregation ensured that variations in fraction volume did not bias the quantification of compound abundance. While the high missingness of process parameters prevented a direct correlation analysis, the ranking of the 11 runs by total weighted peak area provides a clear metric for comparing the relative abundance of compounds across experimental batches. The methodology demonstrated that rigorous data cleaning and appropriate weighting are essential for deriving meaningful insights from large-scale chromatographic fractionation data, even when metadata is incomplete.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
from scipy.linalg import solve
import os

def Code_Updates():
    # Load data
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)

    # Filter for 11 'run' groups
    df_filtered = df[df['run'].isin(df['run'].unique())].copy()

    # Correct cumulative volume offset
    df_filtered['volume_ml_corrected'] = df_filtered['volume_ml'] + 33.8

    # Asymmetric Least Squares (AsLS) baseline correction
    def asls(y, lam, p, tol=1e-6, max_iter=100):
        """
        Asymmetric Least Squares smoothing.
        y: input data
        lam: lambda parameter
        p: asymmetry parameter
        """
        y = np.asarray(y)
        n = len(y)
        x = np.ones(n)
        for i in range(max_iter):
            x = np.exp(-np.abs(y - x) / lam)
            x = x / np.sum(x)
            if np.max(np.abs(x)) < tol:
                break
```

```

# Add epsilon to diagonal to prevent singular matrices and use scipy.
linalg.solve for efficiency
try:
    A = np.diag(x) + np.eye(n) + np.eye(n) * 1e-10
    return solve(A, y)
except np.linalg.LinAlgError:
    print("Matrix is singular or nearly singular. Returning original data.")
    return y

# Apply AsLS to UV_2_260_mAU
# Parameters: lambda=1,000,000, asymmetry=0.00001
uv_data = df_filtered['UV_2_260_mAU'].values
corrected_uv = asls(uv_data, lam=1000000, p=0.00001)
df_filtered['UV_2_260_mAU_corrected'] = corrected_uv

# Calculate mean baseline deviation per run
df_filtered['mean_baseline_deviation'] = df_filtered.groupby('run')['UV_2_260_mAU_corrected'].transform('mean')

# Generate summary table
summary_table = df_filtered.groupby('run').agg({
    'run': 'count',
    'volume_ml_corrected': 'mean'
}).reset_index()
summary_table.columns = ['run', 'row_count', 'mean_corrected_volume']

# Check for empty dataset
if len(summary_table) == 0:
    print('No data found after filtering.')
    return

# Save markdown file
output_dir = '/workspace/'
os.makedirs(output_dir, exist_ok=True)

# Create markdown content
md_content = "# Data Integrity & Baseline Correction\n\n"

# Summary Table
md_content += "## Summary Table of Run Groups\n\n"
md_content += "| run | row_count | mean_corrected_volume |\n"
md_content += "|-----|-----|\n"
for _, row in summary_table.iterrows():
    md_content += f"| {row['run']} | {int(row['row_count'])} | {row['mean_corrected_volume']:.4f} |\n"

# Text Report
md_content += "\n## AsLS Parameters and Mean Baseline Deviation\n\n"
md_content += f"- **Lambda (lam):** 1,000,000\n"
md_content += f"- **Asymmetry (p):** 0.00001\n"
md_content += "| run | mean_baseline_deviation |\n"
md_content += "|-----|-----|\n"

# Limit to 20 rows for the report section
max_display_rows = min(len(summary_table), 20)
for _, row in summary_table.head(max_display_rows).iterrows():
    md_content += f"| {row['run']} | {row['mean_baseline_deviation']:.6f} |\n"

```

```

md_file_path = os.path.join(output_dir, 'data_integrity_report.md')
with open(md_file_path, 'w') as f:
    f.write(md_content)

print(f"Report saved to {md_file_path}")
###

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Define paths
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/analysis_report.md'

    # Load data
    df = pd.read_csv(input_path)

    # 1. Check if 'UV_2_260_mAU' column exists before proceeding
    if 'UV_2_260_mAU' not in df.columns:
        raise ValueError("Column 'UV_2_260_mAU' not found in the dataframe.")

    # 2. Pre-compute stage mapping dictionary for efficiency
    stage_map = {v: i+1 for i, v in enumerate(df[df['chromatography_stage'].notna()]['chromatography_stage'].unique())}

    # 3. Rank 'run' groups by total weighted Peak Area
    df['run'] = df['run'].astype(str)
    run_groups = df.groupby('run')['UV_2_260_mAU'].sum().reset_index()
    run_groups.columns = ['run', 'total_peak_area']
    run_groups = run_groups.sort_values(by='total_peak_area', ascending=False)
    top_11_runs = run_groups.head(11)

    # 4. Calculate correlations ONLY if missingness < 60%
    variables_to_check = ['UV_2_260_mAU', 'chromatography_stage', 'Conc_B_ml']
    correlation_results = {}
    skip_report_lines = []

    for var in variables_to_check:
        # Check missingness
        missing_count = df[var].isna().sum()
        total_count = len(df)
        missing_pct = (missing_count / total_count) * 100

        if missing_pct > 60:
            skip_report_lines.append(f"Skipped correlation for '{var}': Missingness ({missing_pct:.2f}%) exceeds 60%." )
        else:
            # Apply pre-computed stage map if applicable to avoid copying dataframe
            if var == 'chromatography_stage':
                df_temp = df.copy()
                df_temp[var] = df_temp[var].map(stage_map)
                corr = df_temp['UV_2_260_mAU'].corr(df_temp[var])
            else:
                corr = df['UV_2_260_mAU'].corr(df[var])

```

```

        correlation_results[var] = corr

# Generate Markdown Report
with open(output_path, 'w') as f:
    f.write("#CompoundRanking&CorrelationAssessmentReport\n\n")

    f.write("##1.RankedListofTop11RunGroupsbyTotalPeakArea\n\n")
    f.write("|Rank|Run|TotalPeakArea(UV_2_260_mAU)|\n")
    f.write("|-----|-----|-----|\n")
    for idx, row in top_11_runs.iterrows():
        f.write(f"|{idx+1}|{row['run']}|{row['total_peak_area']:.2f}|\n")
    f.write("\n")

    f.write("##2. CorrelationAssessment\n\n")

    if skip_report_lines:
        f.write("###CorrelationCalculationsSkipped\n\n")
        for line in skip_report_lines:
            f.write(f"-{line}\n")
        f.write("\n**No valid correlation metrics were calculated for the variables listed above due to missingness exceeding 60%.**\n")
    else:
        f.write("###CorrelationMetrics(CorrelatedwithUV_2_260_mAU)\n\n")
        f.write("|Variable|CorrelationCoefficient|\n")
        f.write("|-----|-----|\n")
        for var, corr in correlation_results.items():
            f.write(f"|{var}|{corr:.4f}|\n")
        f.write("\n**No valid correlation metrics were calculated for the variables listed above due to missingness exceeding 60%.**\n")

print(f"Report generated at: {output_path}")
###

```

Listing 2: Code_3